

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 5. Conditional Statements

Lecture: 37. Conditional Statements - if...else

Section 1: Lecture Summary

This lecture introduces fundamental concepts related to **conditional statements** in Python programming. It begins by explaining **relational (comparison) operators**, which are used to compare values and return Boolean results (True or False). Next, the lecture covers the basics of **conditional statements**, focusing on the `if` and `else` constructs, explaining their syntax, indentation requirements, and execution flow. Then, a simple program example is demonstrated to check whether a number is positive or negative using these conditional statements. Finally, the lecture discusses the concept of **control flow** in programs, illustrating how statements are executed sequentially in a linear flow or conditionally depending on conditions, represented visually using flowcharts.

Section 2: Key Concepts and Explanations

- Relational (Comparison) Operators

- Used to compare two values.
- Return a **Boolean** value: `True` or `False`.
- Operators include:
 - `<` : less than
 - `<=` : less than or equal to
 - `>` : greater than
 - `>=` : greater than or equal to
 - `==` : equal to (note: **double equals** required in programming, unlike mathematics)
 - `!=` : not equal to

- Example comparisons: `4 < 5` evaluates to `True`; `4 > 5` evaluates to `False`.

- Conditional Statements

- Control program execution based on conditions.
- Syntax structure:

indented block executed if condition is True

indented block executed if condition is False

```
if <condition>:  
    else:
```

- Indentation is **mandatory** in Python (usually four spaces or a tab).
- The condition uses relational operators.
- Only one of the blocks (`if` or `else`) executes depending on the condition's truth value.
- The lecture focused on `if` and `else` statements; `elif` is mentioned but not covered in detail.

- Example Problem: Checking Positive or Negative Number

- Condition: Check if a number `a` is greater than or equal to zero.
- If true, print "Positive".
- Otherwise, print "Negative".

- Control Flow

- Describes the order in which statements are executed in a program.
- **Linear control flow**: statements execute one after another.
- **Conditional control flow**: execution depends on a condition; only part of the code runs.
- Flow can be visualized with **flowcharts**:
 - Rectangles represent statements.
 - Diamonds represent decisions/conditions.

- Arrows show the flow path depending on condition results.

Section 3: Example Code and Use Cases

Example of relational operators evaluation

```
print(4 < 5)          # True
print(4 <= 5)         # True
print(4 > 5)          # False
print(4 >= 5)         # False
print(4 == 5)         # False
print(4 != 5)         # True
```

This code demonstrates how relational operators return Boolean results.

Conditional statement example: Check if number is positive or negative

```
a = 10 # You can change this number to test

if a >= 0:
    print('Positive')
else:
    print('Negative')
```

This program checks whether the number stored in `a` is positive (including zero) or negative and prints the corresponding message.

Testing the same condition with a negative number

```
a = -5

if a >= 0:
    print('Positive')
else:
    print('Negative')
```

Here the same logic is tested with a negative number, demonstrating the `else` branch execution.

Illustrative program flow with conditional execution of statements 5 or 6

Pseudocode representation

```
Statement 1  
Statement 2  
Statement 3  
Statement 4  
  
if <condition>:  
    Statement 5  
else:  
    Statement 6
```

Shows that Statements 1-4 execute linearly, then depending on a condition, either Statement 5 or Statement 6 executes.

Section 4: Key Takeaways

- **Relational operators** compare values and return `True` or `False`.
- Use **double equal signs** (`==`) to test equality in Python, not a single `=`.
- **Conditional statements** (`if` and `else`) execute code blocks selectively based on conditions.
- Python requires **proper indentation** to define the blocks of code under `if` and `else`.
- Control flow can be **linear or conditional**, which affects program execution order.
- Visualizing logic with **flowcharts** aids understanding program control flow.
- Writing correct **conditional expressions** is crucial for expected program behavior.
- Practice is key to mastering writing and using conditional statements effectively.